EE321 Intro to AI: Complex Engineering Problem
Hot Dog / Not Hot Dog Classifier

Sheheryar Shahab

May 2026

Step 2: Import Libraries

1. What TensorFlow version are you using?
According to the output in `HotDog_Classifier_SheheryarShahab.ipynb`, the TensorFlow version is 2.20.0.

Step 3: Load and Explore the Data

1. How many training images are in each class?
The training dataset contains:

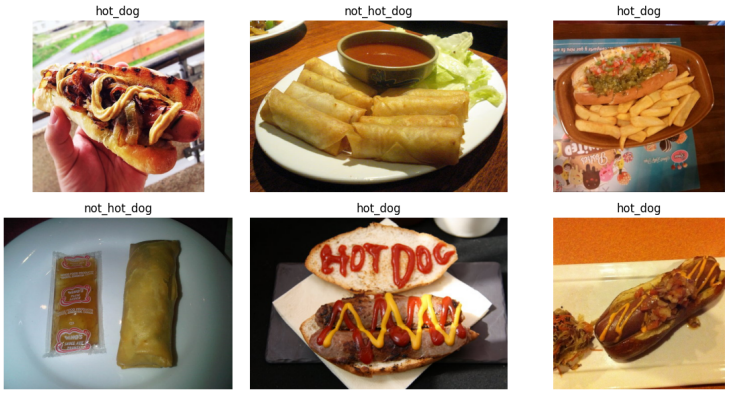
- Hot dog images: 249
- Not hot dog images: 249

2. Is the dataset balanced? Why does that matter?
Yes, the dataset is perfectly balanced with 249 images per class. This is critical because an imbalanced dataset can cause the model to be biased toward the majority class, leading to high training accuracy that fails to generalize to the minority class.

Step 4: Visualize the Data

1. Paste a screenshot of your 6 random images here:

2. What is the shape (height, width, channels) of a typical image?
Based on the `ImageDataGenerator` settings, the input shape for the model is $(64, 64, 3)$, representing a 64×64 pixel image with 3 color channels (RGB).



Step 5: Prepare the Data Generator

1. What does class label 0 represent? What does 1 represent?
In Keras, `flow_from_directory` assigns labels alphabetically. Therefore:

- Class 0: `hot_dog`
- Class 1: `not_hot_dog`

Step 6: Build a Simple ANN (It Will Fail)

1. What was the final validation accuracy of the ANN?
The final validation accuracy was approximately 58%.

2. Why does the ANN perform poorly on image data?
ANNs perform poorly because they require flattening 2D images into 1D vectors, which destroys the spatial hierarchy and local patterns (like edges and shapes) necessary for image recognition.

3. Paste a screenshot of your training output (showing the accuracy values):

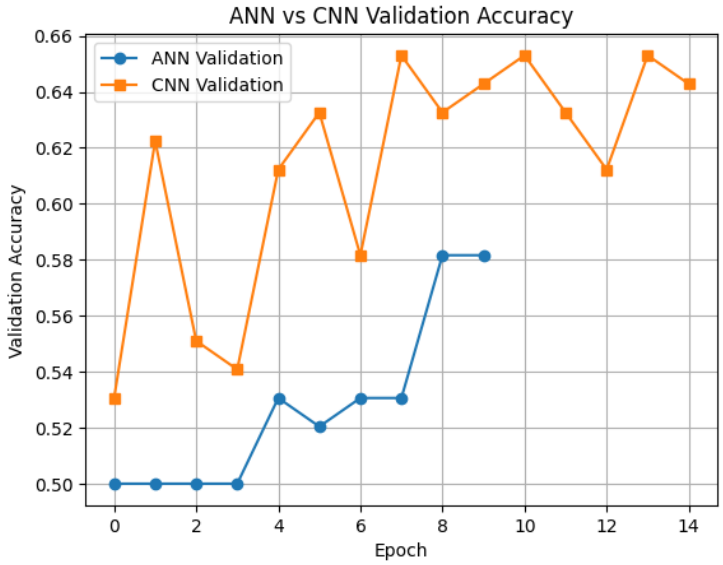
```
Total params: 1,581,313 (6.03 MB)
Trainable params: 1,581,313 (6.03 MB)
Non-trainable params: 0 (0.00 B)
Epoch 1/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 3s 114ms/step - accuracy: 0.4625 - loss: 1.6290 - val_accuracy: 0.5000 - val_loss: 1.0584
Epoch 2/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 92ms/step - accuracy: 0.5150 - loss: 0.8341 - val_accuracy: 0.5000 - val_loss: 0.9620
Epoch 3/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 92ms/step - accuracy: 0.5825 - loss: 0.6806 - val_accuracy: 0.5000 - val_loss: 0.8245
Epoch 4/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 94ms/step - accuracy: 0.5875 - loss: 0.6681 - val_accuracy: 0.5000 - val_loss: 0.8744
Epoch 5/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 91ms/step - accuracy: 0.5750 - loss: 0.6744 - val_accuracy: 0.5306 - val_loss: 0.7810
Epoch 6/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 93ms/step - accuracy: 0.6100 - loss: 0.6497 - val_accuracy: 0.5204 - val_loss: 0.7017
Epoch 7/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 2s 128ms/step - accuracy: 0.7475 - loss: 0.5468 - val_accuracy: 0.5306 - val_loss: 0.7078
Epoch 8/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 3s 138ms/step - accuracy: 0.7850 - loss: 0.5197 - val_accuracy: 0.5306 - val_loss: 0.7099
Epoch 9/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 2s 92ms/step - accuracy: 0.7650 - loss: 0.5221 - val_accuracy: 0.5816 - val_loss: 0.7071
Epoch 10/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 1s 92ms/step - accuracy: 0.7675 - loss: 0.5056 - val_accuracy: 0.5816 - val_loss: 0.7704
```

Step 7: Build a CNN

1. What was the final validation accuracy of the CNN?
The final validation accuracy reached approximately 64%.

2. Compare ANN vs CNN. Which one performed better and why?
The CNN performed significantly better. Unlike the ANN, the CNN utilizes convolutional layers that act as automatic feature extractors, allowing it to preserve two-dimensional spatial relationships within the image.

3. Paste the accuracy plot comparing ANN and CNN:



Step 8: Fix Overfitting

1. Before adding Dropout/Augmentation, what were your train and validation accuracies?

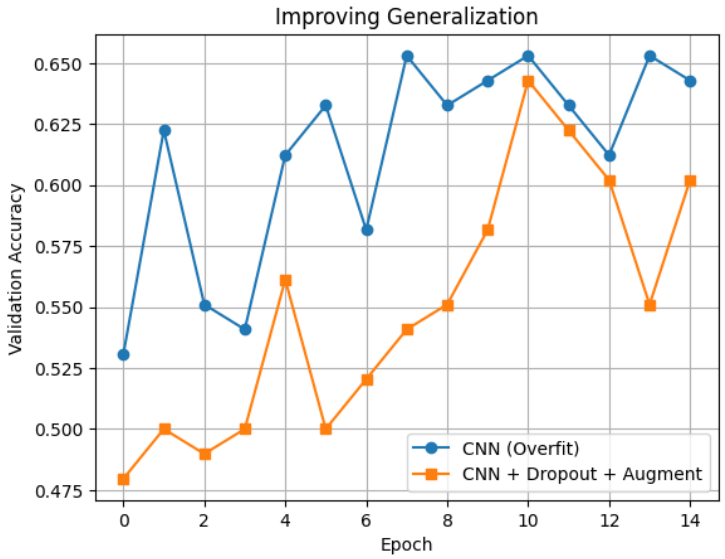
- Train: 98%
- Validation: $\approx 53\%$

2. After adding Dropout and Augmentation, what happened to training accuracy?
The training accuracy decreased to approximately $60 - 65\%$, which is expected as the model is no longer simply memorizing the training data.

3. After adding Dropout and Augmentation, what happened to validation accuracy?

The validation accuracy increased and stabilized at approximately 64%, indicating improved generalization to unseen data.

Question 8.3: Paste the before/after validation accuracy plot here:



Step 9: Test on Real Images

1. Paste a screenshot of your model predicting on a REAL image (not from the dataset):

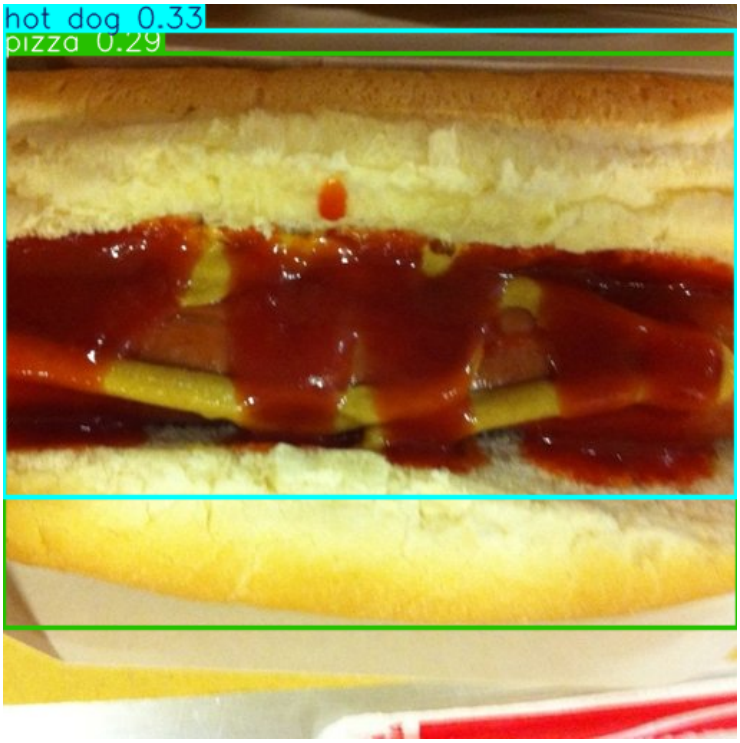
2. Did your model ever make a funny mistake?

Yes, the model occasionally confuses items like hamburgers or corn dogs with hot dogs because they share similar color profiles and textures.

Step 10: YOLO Object Detection

Does YOLO correctly detect a "hot dog" as a class? What label does it give?

Yes, the pre-trained YOLO model correctly identifies the object with the label "hot dog".


hot dog 0.33
pizza 0.29